

CHAPTER 12

Software Quality

ACRONYMS

CI/CD	Continuous Integration/ Continuous Delivery
CoSQ	Cost of Software Quality
COTS	Commercial Off-The-Shelf
FMEA	Failure Mode and Effects Analysis
FTA	Fault Tree Analysis
IV&V	Independent Verification and Validation
PDCA	Plan-Do-Check-Act
PSP	Personal Software Process
QFD	Quality Function Deployment
RCA	Root Cause Analysis
SCM	Software Configuration Management
SQA	Software Quality Assurance
SQAP	Software Quality Assurance Plan
SQC	Software Quality Control
SQM	Software Quality Management
V&V	Verification and Validation

INTRODUCTION

What is software quality, and why is it so important that it is included in many knowledge areas (KAs) of the *SWEBOK Guide*? One reason is that the term *software quality* is overloaded. Software quality may refer to the desirable characteristics of software products, to the extent to which a particular software product has those characteristics (software product quality), and to the processes, tools and techniques used to achieve those characteristics (software process quality). Over the years, authors and organizations have defined

the term *quality* differently. Phil Crosby defined quality as “conformance to requirements” [2]. Watts Humphrey referred to it as “achieving excellent levels of “fitness for use” [3]. Meanwhile, IBM coined the phrase “market-driven quality,” where the “customer is the final arbiter” [4]. Finally, fitness for purpose is also a term that refers to software quality. Fitness for purpose is the suitability of a product, system, or service for use by the intended users, for the intended use, in the intended situations, and intended environmental conditions.

More recently, software (product) quality has been defined as the “capability of a software product to satisfy stated and implied needs under specified conditions” [4] and as “the degree to which a software product meets established requirements; however, quality depends upon the degree to which those established requirements accurately represent stakeholder needs, wants, and expectations” [6]. Both definitions embrace the premise of conformance to requirements. Neither refers to different types of requirements (requirements categorized according to functionality, reliability, performance, dependability, or any other characteristic). Significantly, however, these definitions emphasize that quality is an important characteristic of requirements.

These definitions also illustrate another reason for the recurring discussions about software quality throughout the *SWEBOK Guide* — the often-unclear distinction between *software quality* and *software quality requirements* (“the *-ilities*” is a common shorthand for these terms). Software quality requirements (Quality of Service Constraints in the Software Requirements KA) are attributes of (or constraints on) functional requirements

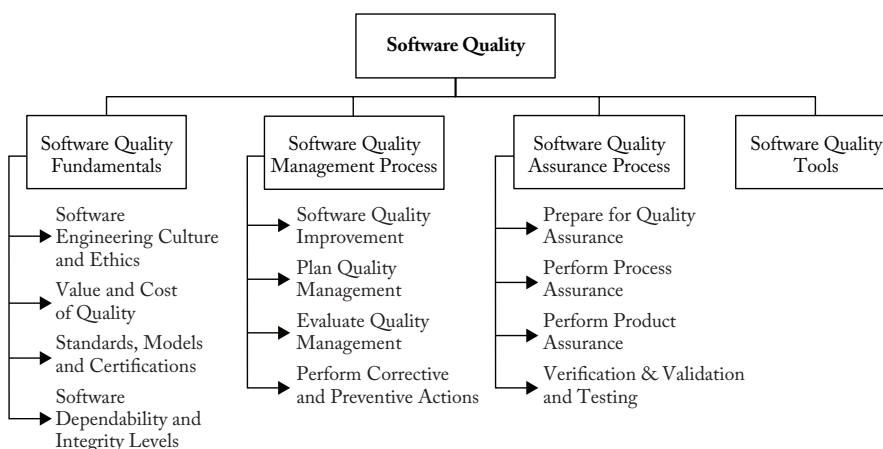


Figure 12.1. Breakdown of Topics for Software Quality

(what the system does). Software requirements may also specify resource use, a communication protocol, or many other characteristics (Technology Constraints in the Software Requirements KA). This KA attempts to clarify requirements by using *software quality* in the broadest sense from the definitions above and by using *software quality requirements* as constraints on functional requirements. Software product quality is achieved by conforming to all requirements regardless of specified characteristics or grouping or naming of requirements.

Software quality is also discussed in many other SWEBOK Guide Knowledge Areas because it is a basic concept of a software engineering effort. The primary goal for all engineered products is to deliver maximum stakeholder value while balancing the constraints of development, maintenance, and operational cost, sometimes characterized as *fitness for use*. Stakeholder value is expressed in requirements. For software products, stakeholders could value price (what they pay for the product), lead time (how fast they get the product), and quality. (See the Software Requirements KA for a broader discussion of this.)

The software process quality aspect, which is implied by the above, must be made explicit. The quality of a software process can also be

observed in process characteristics such as efficiency, effectiveness, usability, and learnability. Defects in that process will likely show up as defects in the resulting software product, as well.

Finally, the Agile and DevOps movements aim at improving the software process and product quality through compliance by promoting quick iteration feedback loops and eliminating organizational silos by collocating users and software engineers. Other practices like pair programming and the automation of development, testing, and operations services also bring value, improve efficiency, and can detect defects early. (Refer to Software Engineering Operations KA for more information about DevOps and Agile life cycles.)

This KA provides an overview of practices, tools, and techniques for understanding software quality and planning and appraising the state of software quality during development, maintenance and operation, from both a software product perspective and a software process perspective.

BREAKDOWN OF TOPICS FOR SOFTWARE QUALITY

The breakdown of topics for the Software Quality KA is presented in Figure 12.1.

1. Software Quality Fundamentals

Agreeing on what constitutes software quality for all stakeholders and communicating that agreement to software engineers requires that the many aspects of quality be formally defined and communicated. The main challenges the software engineer faces to ensure quality include the following:

- Difficulty in clearly defining requirements;
- Maintaining effective communication with the client/user;
- Deviations from specifications;
- Architecture and design errors;
- Coding errors;
- Noncompliance with current processes/procedures;
- Inadequate work product reviews and tests;
- Documentation errors.

Software quality is defined as “The degree to which a software product meets established requirements; however, quality depends upon the degree to which those established requirements accurately represent stakeholder needs, wants, and expectations.” [6]. It is further defined “by the degree to which a software product meets established requirements; however, quality depends upon the degree to which those established requirements accurately represent stakeholder needs, wants, and expectations” [6]. Quality often means the absence of defects. The word *defect* is overloaded with too many meanings, as engineers and others use the word to refer to all different types of anomalies. However, different engineering cultures and standards often understand “defect” and other terms as having more specific meanings. To avoid confusion, software engineers should use the meaning provided by their standards [14]:

- *Error*: “Human action that produces an incorrect result.” Also called *human error*;
- *Defect*: (*synonym of a fault*) An “imperfection or deficiency in a work product where that work product does not meet its requirements or specifications and needs

to be either repaired or replaced.” A defect is inserted when a person developing the software makes an error. It hides in the software until (and if) it is discovered;

- *Failure*: The “termination of the ability of a system to perform a required function or its inability to perform within previously specified limits; an externally visible deviation from the system’s specification.” A failure is produced when the software executes a defect.

A software engineer should understand software quality concepts, characteristics, and values and their application to the many development, maintenance, and operation activities. The software requirements are expected to define the required software quality attributes. Furthermore, software requirements influence the measurement methods and acceptance criteria for assessing how the software and related work products achieve the desired quality levels. Software quality should be planned early and assessed at many milestones during the software life cycle. Finally, how to adapt software quality assurance (SQA) activities to accommodate different life cycles, for example Agile software development is presented in detail in the Institute of Electrical and Electronics Engineers (IEEE) Standard 730:2014 [6].

1.1. Software Engineering Culture and Ethics [1*, c1s1.6; c2s3] [5*]

An organization’s culture affects how software engineers influence software quality. As Iberle [19] explains, software engineering practices vary depending on the business model (e.g., custom, mass-market, commercial, firmware) and the industry where the software engineers work. Software engineers are expected to share a commitment to software quality in the context of their industry and as part of their culture. A healthy software engineering culture includes many characteristics, such as the understanding that trade-offs among cost, schedule and quality are a basic tenet of any product’s engineering. A strong software engineering ethic assumes

that engineers accurately report information, conditions and outcomes related to quality. Ethics also play a significant role in software quality in the professional culture of engineering, and in the attitudes of software engineers. The IEEE Computer Society and the Association for Computing Machinery (ACM) have developed a code of ethics and professional practice. (See Codes of Ethics and Professional Conduct in the Software Engineering Professional Practice KA.)

1.2. Value and Costs of Quality [1*, c2s2]

One major factor driving resistance to implementing SQA is its perceived high cost. However, not implementing basic SQA activities can be costly as well. Software engineers should inform their administration of the risks they take when they are not fully committed to quality. This can be done by explaining the cost of software quality concepts to management. Cost of software quality (CoSQ) is defined as the sum of the following project costs:

- Implementation cost of planning and construction activities (e.g., planning, designing, development);
- Prevention cost of activities (process improvement, tools, training);
- Appraisal costs activities for defect detection (e.g., reviews, audits, testing);
- Nonconformance and rework costs (internal failure cost and external failure cost).

The CoSQ can be broken down into two top-level categories: *conformance cost* and *non-conformance cost*. Conformance cost is the total of all investments in error and defect detection (*appraisal*) and prevention activities. Appraisal costs arise from project activities that are intended to find errors and defects. These include testing (as detailed in the Software Testing KA) and reviews and audits (as detailed later in this KA). Appraisal costs extend to subcontracted software suppliers, if any. Prevention costs include investments in software process improvement (SPI) efforts,

quality infrastructure, quality tools, work product templates and training. These costs might not be specific to a project; they often span the larger organization.

Nonconformance cost is the total of all spending dealing with errors and defects that have been detected. Pre-delivery costs are those incurred to repair errors and defects found during appraisal activities and discovered before the software product is delivered to the customer. Post-Delivery costs include those incurred responding to software failures discovered after delivery to the customer. External costs include the rework needed to repair and test an updated release. External costs include rework and repair of the unintended and uncompensated side effects or consequences of defects. However, the financial impact on the customer who encounters a failure is just as important. For example, the customer's lost productivity, lost data, and potential loss of reputation in the marketplace must be acknowledged and accounted for. Beyond the impact on the customer, low-quality software can also impact the public and the environment. Software engineers should seek the optimal CoSQ—the minimal total cost for a specified quality level.

1.3. Standards, Models, and Certifications [1*, c4] [7, c24s2]

Sound use of software engineering, software standards, and software process assessment and improvement improves software quality. One of the key general software engineering standards is ISO/IEC/IEEE 12207:2017, which describes the software life cycle processes. Foremost, software engineers should know the key software engineering standards that apply to their specific industry. As Iberle discussed [19], the practices software engineers use vary greatly depending on the industry, business model and organizational culture where they work. For example, IEEE 1228:1994 Standard for Software Safety Plans and IEEE 1633:2016 Recommended Practice on Software Reliability target industries where safety and reliability are important.

The Plan-Do-Check-Act (PDCA) paradigm differs from standards in that it often proposes “best practices” for software engineers from a specific perspective. (Refer to the Software Engineering Process KA for more information about the PDCA paradigm for software.)

Other industry “best practices” models such as the Control Objectives for Information and Related Technologies (COBIT) for information technology governance [27], the Project Management Body of Knowledge (PMBOK®) for project management [25], the Business Analysis Body of Knowledge (BABOK®) [28], the Capability Maturity Model Integration (CMMI) [29] and The Open Group Architecture Framework (TOGAF) propose software related practices that can improve the quality of software processes and products [30]. Software organizations can also consider the possible advantages of obtaining registrations or certifications (e.g., ISO 9001 for quality [10], ISO 27001 for security [31], and ISO 20000 for operations [32]), and software engineers can also obtain Scrum and Scaled Agile Framework® (SAFe®) certifications for Agile processes [22]. The use of these models and certifications have been shown to augment stakeholders’ confidence that the software engineers’ knowledge and skills are up to date and recognized internationally.

1.4. *Software Dependability and Integrity Levels* [1*, c4s8, c7s3.3] [11]

Software-intensive and safety-critical systems are those in which a system failure could harm human life, other living things, physical structures, or the environment. The software in these systems is considered safety-critical and requires the use of systematic methods and tools to ensure its high level of quality. A growing number of industries are using safety-critical software, including transportation, chemical and nuclear, and medical. Software failure in these systems could have catastrophic effects. Engineers use industry standards, such as software considerations in airborne systems and equipment certification

DO-178C [8] and railway applications EN 50128 [18], and emerging processes, tools, and techniques to develop safety-critical software more safely. These standards, tools and techniques reduce the risk of injecting faults into the software and thus improve software availability, reliability, and maintainability. Software engineers and their managers must understand the threats and issues and develop the skills needed to anticipate and prevent accidents before they occur [15].

Safety-critical software can be categorized as *direct* or *indirect*. Direct software is embedded in a safety-critical system, such as an aircraft’s flight control computer. Indirect software includes software applications used to develop safety-critical software. Indirect software is also included in software engineering environments and software test environments.

Three complementary techniques for reducing failure risk are avoidance, detection and removal, and damage limitation. These techniques impact software functional requirements, performance requirements and development processes. Increasing risk implies increasing SQA and more rigorous review techniques such as inspections [16]. Higher risk levels may necessitate more thorough inspections of requirements, design, and code, or the use of more formal verification and validation techniques. Another technique for managing and controlling software risk is building assurance cases. An *assurance case* is a reasoned, auditable artifact created to support the contention that its claim or claims are satisfied. It contains the following relationships: one or more claims about properties, arguments that logically link the evidence and any assumptions to the claims, and a body of evidence and assumptions supporting these arguments [9].

1.4.1. *Dependability* [7, c10]

In cases where system failure may have severe consequences, overall dependability (e.g., hardware, software, and human or operational dependability) is the main quality requirement, aside from basic software functionality,

for the following reasons: System failures affect many people; users often reject systems that are unreliable, unsafe, or insecure; system failure costs could be important; and undependable systems might cause information loss. Many standards address different perspectives of dependability, such as reliability and availability. System and software dependability regroups several related quality characteristics: availability, reliability, maintainability and supportability, safety and security [21]. When developing dependable software, engineers can apply tools and techniques to reduce the risk of injecting faults into the intermediate deliverables or the final software product. They can use static, dynamic, or formal methods for verification and validation (V&V), and testing processes, as well as other specialized techniques, methods, and tools to identify defects that affect dependability as early as possible in the software life cycle [7*, c10.5]. Additionally, they may have to incorporate specific mechanisms into the software to guard against external attacks and to tolerate faults during its operation.

1.4.2. Integrity Levels of Software [1*, c4s8, c7s3.2] [11]

Defining integrity levels is a method of risk management. An *integrity level* is “a value representing project-unique characteristics (e.g., complexity, criticality, risk, safety level, security level, desired performance, and reliability) that define the importance of the system, software, or hardware to the user” [11]. The characteristics used to determine software integrity level vary depending on the intended application and use of the system. The software is a part of the system, and its integrity level is determined as a part of that system.

The assigned software integrity levels might change as the software evolves. Design, coding, procedural and technology features implemented in the system or software can raise or lower the assigned software integrity levels. The software integrity levels established for a project result from agreements among the acquirer, supplier, developer, and

independent assurance authorities. A *software integrity level scheme* is used to determine software integrity levels [11].

Certain safety-critical industries, such as avionics, railways, nuclear power, medical devices and many others, industry-specific guidance can require a certain level of independence for software quality activities and can assign minimum V&V techniques to be used by integrity level (examples of such techniques are: usability analysis, algorithm analysis, boundary value analysis, data flow analysis, walk-through review [11][26]).

2. Software Quality Management Process

Software quality management (SQM) is concerned with “coordinated activities to direct and control an organization with regard to software quality” [6]. The purpose of the Quality Management process is to assure that products, services, and implementations of the quality management process meet organizational and project quality objectives and achieve customer satisfaction.

An important concept of SQM is the design and upkeep of a Quality Management System (QMS). ISO90003 [26] interprets ISO9001 concepts for the software industry.

QMS defines processes, process owners, requirements for the processes, measurements of the processes and their outputs, and feedback channels throughout the whole software life cycle. A QMS comprises many key activities: SQA, V&V, reviews and audits, software configuration management (SCM), and requires policies, procedures, and processes to ensure that everyone involved understands what is expected in terms of software process and product quality. For a QMS to be effective, management support is imperative. Management support implies that projects are trained to the QMS requirement and have enough resources to achieve the quality goal defined for it. Management sponsorship should be solicited frequently during software project review to ensure software quality activities are executed and nonconformities addressed.

For a software project, software quality processes consist of tasks and techniques to indicate how software plans (e.g., software management, development, quality management or configuration management plans) are implemented and how well the intermediate and final products meet their specified requirements. Results from these tasks are assembled in reports for management. SQM process management is tasked with ensuring that the report results are accurate and acted upon.

Risk management can also play an important role in delivering quality software. Incorporating disciplined risk analysis and management techniques into the software life cycle processes can help improve product quality. (See the Software Engineering Management KA for related material on risk management.)

2.1. *Software Quality Improvement*

[1*, c9 and c9s9] [2] [3]

Software quality improvement (SQI) is done using many different approaches within the software industry, including software process improvement (SPI), Six Sigma, Lean, and Kaizen, just to name a few. For example, the SPI activities seek to improve process effectiveness, efficiency, and other characteristics to improve software quality. For example, although SPI could be included in any of the first three categories, many organizations organize SPI into a separate category that might span many projects.

Software product quality can be improved using Lean principles as well as an iterative process of continual improvement, which includes management control, coordination of activities, and feedback from many concurrent processes: (1) the process of improving the software life cycle processes; (2) the process of fault/defect categorization, detection, removal, and prevention; and (3) a personal improvement process.

The theory and concepts behind quality improvement — such as building quality through the prevention and early detection of defects, continual improvement, and

stakeholder focus — are also pertinent to software engineering. Experts in quality have stated that a product's quality is directly linked to the quality of the process used to create it. Improvement models such as the Plan-Do-Check-Act (PDCA) improvement cycle, evolutionary delivery, Kaizen, and techniques like quality function deployment (QFD) offer ways to specify quality objectives and determine whether they are met.

Since software engineering is a complex process, it cannot be reduced to a cookbook of procedures. To complement the process and tools improvement movement, Humphrey [32] proposed the personal software process (PSP) for software engineers to also assess their skills and knowledge constantly and continually improve them as well.

2.2. *Plan Quality Management* [1*, c13]

Software quality planning includes defining the software quality process to be used and determining which quality standards and models are to be used, defining specific quality goals, estimating the effort to be used to achieve each goal; and deciding at what activities the software quality activity should take place.

First, the software organization must commit to quality by establishing their quality management system (QMS), which includes quality management policies, objectives, and procedures. This requires that the responsibility and authority for implementing the QMS are assigned and that they are independent of current project management teams.

An approved organizational policy, about software quality, helps in guiding projects and products development decisions as well as behavior of personnel. Software engineers should promote the use of formally approved processes and procedures that implement the quality policy and explain the roles, activities to be executed and the expected results of key software engineering activities. Consequently, for a QMS to be used in improvement its processes should be documented with its user in mind and identify where quality controls are

to be verified. Procedures explain in detail what steps are taken to execute a specific activity.

2.3. *Evaluate Quality Management*

Once the QMS is in place, the ISO/IEC Technical Specification TS 33061:2021 [23] Standard defines a process assessment model for software life cycle processes using five process capabilities levels (from level 0: incomplete to level 5: optimizing process). Additionally, software engineers can assess the maturity of their QMS activities in their software projects using the IEEE 730:2014 Standard guidance [6]. Management sponsorship supports process and product evaluations. The evaluation findings feed into an improvement program for identifying detailed actions and improvement projects to be addressed in a feasible time frame. Periodically, the software engineers will gather and analyze quality assurance evaluation results. This can be achieved by looking at quality measures and defect characterization produced by the projects.

2.3.1. *Software Quality Measurement* [1*, c10] [7, 24s5]

Software quality measurements are used to support decision-making. With the increasing sophistication of software, quality questions go beyond whether the software works to how well it achieves measurable quality goals. Quantifying some attribute of software can help engineers evaluate its quality or the quality of its process. (Process measurement is described in detail in the Software Engineering KA.)

Software quality measurement helps engineers make determinations about software quality (because models of software product quality include measures to determine the degree to which the software product achieves quality goals); managerial questions about effort, cost, and schedule; when to stop testing and release a product (see Test-Related Measures in the Software Testing KA); and the efficacy of process improvement efforts.

The CoSQ assurance activities are an issue frequently raised in deciding how a project or a software development and maintenance organization should be organized. Often, generic models of cost are used; these models are based on when a defect is found and how much effort it takes to fix the defect relative to finding the defect earlier in development. Software quality measurement data collected internally may offer a better picture of cost within the project or organization. Although the software quality measurement data may be useful by itself (e.g., the number of defective requirements or the proportion of defective requirements), mathematical and graphical techniques can help project stakeholders interpret the measures. (See the Engineering Foundations KA and the Mathematical Foundations KA.) These techniques include the following:

- Descriptive statistics-based analysis (e.g., Pareto analysis, run charts, scatter plots, normal distribution);
- Statistical tests (e.g., the binomial test, chi-squared test);
- Trend analysis (e.g., control charts; see *The Quality Toolbox* in Further Readings);
- Prediction (e.g., reliability models).

Descriptive statistics-based techniques and tests often provide a snapshot of the more troublesome areas of the software product under examination. The resulting charts and graphs are visualization aids decision-makers can use to focus resources and conduct process improvements where they seem most needed. Results from trend analysis may indicate that a schedule is slipping or that certain classes of faults may become more likely unless some corrective action is taken in development. The predictive techniques help estimate testing effort and schedule and predict failures. (More discussion on measurement in general appears in the Software Engineering Process and Software Engineering Management KAs. More specific information on testing measurement is presented in the Software Testing KA.) Software quality measurement also

includes measuring defect occurrences and applying statistical methods to understand what types of defects occur most frequently. Three widely used software quality measurements are error density (number of errors per unit size of documents/software), defect density (number of defects found divided by the size of the software), and failure rate (mean time to failure). Reliability models are built from failure data collected during software testing or from software in service and thus can be used to estimate the probability of future failures and assist in decisions about when to stop testing. This information can be used in SPI to determine methods to prevent, reduce or eliminate defect recurrence. The information also helps engineers understand trends, how well detection and containment techniques are working, and how well the development and maintenance processes are progressing. They can use these measurement methods to develop defect profiles for a specific application domain. Then, for the next software project within that organization, the profiles can be used to guide the SQM processes — that is, to focus effort on where problems are most likely to occur. Similarly, benchmarks, or defect counts typical of that domain, may help engineers determine when the product is ready for delivery. (Discussion about using measurement data to improve development and maintenance processes appears in the Software Engineering Management and Software Engineering Process KAs.)

2.4. *Perform Corrective and Preventive Actions*

It is important that when quality management objectives are not met, corrective actions be documented and submitted so that the QMS be improved to prevent problem from reoccurring in future software projects. This requires that project participants have a way of reporting software engineering process and tools problems to an independent organization that will document and monitor the progress of the corrective actions and inform the relevant stakeholders.

2.4.1. *Defect Characterization* [1*, c1s3]

To help in the elimination of the cause or causes of an existing nonconformity or undesirable situation to prevent recurrence, software engineers can use software quality control (SQC) techniques to find errors, defects, and failures in their processes and products. When tracking errors, defects and failures, the software engineer is interested in the number and types of incidents. Numbers alone, without classification, might be insufficient to help in identifying the underlying causes and thus to prevent them in the future. Therefore, software engineers should establish a meaningful defect classification taxonomy to describe and categorize such anomalies. One probable action resulting from peer reviews and testing findings is to remove these errors and defects early from the work product under examination.

Other SQM activities attempt to eliminate their causes (e.g., root cause analysis (RCA)). RCA activities include analyzing and summarizing the findings to find root causes and using measurement techniques to improve the software engineering processes, techniques and tools. (Process improvement is primarily discussed in the Software Engineering Process KA. RCA is further discussed in the Engineering Foundations KA.)

Data on errors and defects found during SQA and control techniques may be lost unless they are recorded. For some techniques (e.g., peer reviews and inspections), software engineers are present to record such data and to address issues and make decisions. In addition, when automated tools are used (see Topic 4, Software Quality Tools), the tool output may provide defect trends reports that can be provided to the organization's management.

3. **Software Quality Assurance Process**

3.1. *Prepare for Quality Assurance* [1*, c1s5, c4s6] [6]

Software quality assurance (SQA) is defined as “a set of activities that define and assess the

adequacy of software processes to provide evidence that establishes confidence that the software processes are appropriate for and produce software products of suitable quality for their intended purposes.” To correct a common misunderstanding, SQA is not only testing of a software. A key attribute of SQA, in critical systems, is the objectivity of the SQA function concerning the quality of a software product. In this case, the SQA function might also be organizationally independent of the project; that is, free from technical, managerial, and financial pressures [6]. SQA has two aspects: product assurance and process assurance, which are introduced in Section 2.3.

The software quality plan (in some industry sectors, it is termed the software quality assurance plan (SQAP)) defines the activities and tasks used to ensure that software developed for a specific product satisfies the project’s established requirements and user needs within project cost and schedule constraints and is commensurate with project risks. The SQAP first ensures that quality targets are clearly defined and understood.

The SQAP’s quality activities and tasks are specified, along with their costs, resource requirements, objectives, and schedule in relation to related objectives, in the software engineering management, software development and software maintenance plans. The SQAP identifies documents, standards, practices, and conventions governing the project and how these items are checked and monitored to ensure adequacy and compliance. The SQAP also identifies measures; statistical techniques; procedures for problem reporting and corrective action; resources such as tools, techniques, and methodologies; security for physical media; training; and SQA reporting and documentation. Moreover, the SQAP addresses the SQA activities of any other type of activity described in the software plans — such as procurement of supplier software for the project, commercial off-the-shelf (COTS) software installation and service after software delivery. It can also contain acceptance criteria and reporting and management activities that are critical to software quality. The

SQA plan should not conflict with the software configuration management plan or any other relevant project planning artifact. Moreover, they should be considered complementary activities (for process SQAP the SCM Process Audit and the Testing activities for SCM Functional Audit).

Software quality encompasses several perspectives: the software process quality, the software end-product quality and the software work products (also called *intermediary products*) quality. The next sections cover each perspective of software quality knowledge a software engineer must have.

3.2. Perform Process Assurance

[1*, c3s2-3, c4s6.1.3, c8, c9] [7, c25]

Crosby [2] and Humphrey [3] have demonstrated that software quality management (SQM) and software engineering process quality have a direct effect on the quality of the final software product. (Models and criteria that evaluate and improve the capabilities of software organizations are primarily project organization and management considerations and, as such, are covered in the Software Engineering Management and Software Engineering Process KAs.) ISO 9001 [10] proposes another process quality perspective, where a management system that oversees the processes’ actors, activities, controls, input, and outputs ensures the quality of outputs (e.g., work products and final product). A *management system* is defined as a “set of interrelated or interacting elements of an organization to establish policies and objectives, and processes to achieve those objectives” [10]. This perspective requires software engineering organizations to take the time to describe their policies, processes, and procedures with enough detail that software engineer roles and responsibilities are clear during life cycle activities (as detailed in the Software Engineering Process KA).

SQA activities, listed in IEEE730:2014 [6], describe the many quality assurance activities that should be conducted early in a software project’s life cycle to ensure quality. Software

engineers should be aware of the need to plan and execute SQA activities at certain project milestones and keep records of their execution. These activities consist of document and code reviews as well as verification and validation (V&V) activities, including testing (as detailed in Section 3.4 of this KA), which evaluate the output of a process's compliance with its requirements and specifications.

Finally, software configuration management (SCM) is an important activity to ensure the quality of work products and software. *Configuration management* is defined as the “discipline applying technical and administrative direction and surveillance to:

- identify and document the functional and physical characteristics of a configuration item;
- control changes to those characteristics;
- record and report change processing and implementation status;
- verify compliance with specified requirements.”

Software engineers should identify which work products and software artifacts require configuration management. In addition, they should be familiar with source code versioning processes, which involve keeping track of baselined and incremental versions of the software and ensuring that changes different developers make do not interfere with one another, and they should know how to operate the version control tool kit. (Refer to the Software Configuration Management KA for more information about this process.)

3.3. Perform Product Assurance

[1*, s3.2–s3.3] [7, c4, s6.1.2]

First, the software engineer must determine the real purpose of the software to be designed and constructed. Stakeholder requirements are paramount here. They include quality requirements (called *Quality of Service Constraints* in the Software Requirements KA) and functional requirements. Thus, software engineers

are responsible for eliciting quality requirements that might not be explicit at the outset and for understanding their importance and the difficulty in defining them, measuring them, and establishing them for final acceptance. Software engineers should understand how to define quality requirements as well as their quality targets to ensure they can effectively be measured at the acceptance stage of the project. During the project planning, software engineers must keep these quality requirements in mind. They must also anticipate potential additional development costs if attributes such as safety, security and dependability are important.

What constitutes a software product's many measurable quality characteristics is described in ISO/IEC 25010:2011 [4]. This standard proposes several software product quality models, consisting of characteristics and sub-characteristics, for software product quality and software quality in use. Another is IEEE 982.1:2005 Standard Dictionary of Measures to Produce Reliable Software. These software characteristics are commonly called *product quality requirements*, which are nonfunctional software requirements [7*, c4, s6.1.2]. Software engineers should know the many software characteristics that can be planned, implemented, and measured during software construction (e.g., functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability). Software engineers should also know that certain quality characteristics have conflicting impacts. For example, trying to augment the security characteristic by encrypting data might adversely affect the performance characteristic. This standard also proposes a general data quality model that focuses on data quality as part of a computer system and defines quality characteristics for target data used by humans and systems.

Another software product quality perspective is the quality of work products. The term *work product* means any artifact resulting from a process used to create the final software product. Work products include system/

subsystem specifications, software requirements specifications for a system's software components, software design descriptions, source code, software test documentation and test reports. Sound engineering practice requires that intermediate work products relevant to quality be evaluated using work product reviews and inspections (discussed later in this chapter) throughout the software engineering process.

3.4. *V&V and Testing* [1*, c7] [11]

Verification ensures that the product is built correctly in that the output products of a life cycle phase meet the specifications imposed on them in previous phases. Verification is defined as “the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase” [11]. Alternatively, *validation* ensures that the right product is built — the product fulfills its specific intended purpose. It is defined as “the process of evaluating a system or component during or at the end of the development process to determine whether it satisfies specified requirements.”

The purpose of V&V is to help the development organization build quality into the software throughout the development life cycle. V&V includes software testing tasks. Software testing is a necessary activity to ensure product quality. However, in most cases, software testing is insufficient to establish confidence that the software fits its intended use. V&V tasks listed in IEEE Standard 1012:2016 [11] objectively assess products and processes throughout the life cycle. This assessment demonstrates whether the requirements are correct, complete, accurate, consistent, and testable. The verification process and the validation process should begin early in development or maintenance. This prevents defects late in the life cycle, which would incur rework and significantly increase costs. Software engineers should identify the product integrity level and ensure the minimum V&V tasks are assigned for key

product features concerning both the product's immediate predecessor and the planned specifications. Optional V&V tasks are also listed and can improve software product quality. Keeping a record of the traceability among software work products can help augment the quality of the V&V activities. *Traceability* is defined as the “ability to trace the history, application or location of an object” [14].

Early planning of V&V activities ensures that each resource, role, and responsibility is clearly assigned. The resulting V&V plan documents the various resources and their roles and SQA activities, as well as the techniques and tools to be used. Software engineers should choose and apply the proper V&V task depending on the software integrity level. (Refer to Section 1.4.2). V&V can also be executed by an independent organization for very critical software. Independent verification and validation (IV&V) are defined as “V&V performed by an organization that is technically, managerially, and financially independent of the development organization” [11].

Software V&V tasks can be sorted into static, dynamic and formal tasks [20]. Dynamic techniques involve executing the software; static techniques involve analyzing documents and source code but not executing the software; formal techniques use mathematics and formal specification languages.

There are no strong boundaries between “Static analysis techniques”, “dynamic analysis techniques” and “formal analysis techniques”. For example, static and dynamic analysis techniques usually have a strong formal background such as data-flow analysis or model checking.

3.4.1. *Static Analysis Techniques*

Static analysis techniques analyze a work product's content and structure (including requirements, interface specifications, designs, and models) without executing the software. The only way to detect non-executable code is through static analysis as no dynamic test can verify that. Static techniques

can be executed manually or with the help of a tool. For example, code reading, peer review of a work product, and static analysis of source code control flow are considered static techniques because they do not involve executing the software code.

Section 3.4.5 will show that review and audit processes are considered static analysis activities, meaning that no software or models are executed. Instead, they examine software engineering artifacts (also called intermediary or work products) concerning standards established by the organization or project for those artifacts.

3.4.2. *Dynamic Analysis Techniques*

Dynamic analysis techniques involve executing or simulating the software code, looking for errors and defects. Different dynamic techniques are performed throughout software development, maintenance, and operation. Generally, these are testing techniques, but simulation, model analysis and model checking are considered dynamic analysis techniques. (See the Software Engineering Models and Methods KA.) In addition, black box testing is considered a dynamic analysis technique, as the software engineer analyzes the output received following the entry of inputs. (See the Software Testing KA.)

3.4.3. *Formal Analysis Techniques* [7*, c10s5]

Formal analysis techniques (also called *formal methods*) are “mathematical approaches to software development where you define a formal model of the software. You may then formally analyze this model to search for errors and inconsistencies” [7*, c10s5]. Sometimes, the software requirements may be written using a more formal specification language known as formal methods. They have mostly been used to verify crucial parts of critical systems, such as specific security and safety requirements. (See also Formal Methods in the Software Engineering Models and Methods KA.) Different groups may perform testing

during software development, including groups independent of the development team. The Software Testing KA is devoted entirely to this subject.

3.4.4. *Software Quality Control and Testing* [1*, c7s10]

Testing is considered an important product quality control activity included in a software development project’s V&V processes. Quality Control is “a set of activities that measure, evaluate and report on the quality of software project artifacts throughout the project life cycle” [25]. Software testing is one of many verification activities that confirm that software development output meets input requirements. IEEE 730:2014 [6] lists the many testing and retesting activities software engineers should plan, execute, and record. It also recommends that testing completion criteria be set. Software engineers should plan the testing activities, including levels, techniques, measures, and tools. A software quality engineering team, for critical systems, should particularly be involved in qualifying software products prior to its delivery (i) either for further integration or (ii) for operations in target computing environment; as an independent test and evaluation activity, without involving development team members in the process. (Refer to the Testing KA for details about the knowledge software engineers should have about software testing.)

3.4.5. *Technical Reviews and Audits* [1*, c5, c6] [23, s4, s5]

SQC techniques, for assessing the quality of a software, were presented in section 2.4.1. For the other artifacts, product quality control is assessed using reviews and inspections of these work products. These SQC activities are planned and executed during development, maintenance, and operations activities [17].

Reviews are valuable because they can identify issues early in development or even before a component is designed. Fixing a

defect in a component that has been coded is much more expensive than catching it beforehand. Peer reviews are defined as “the review of work products performed by peers during development of the work products to identify defects for removal” [14]. For example, during software development, a code review (often done by using a pull request technique/tool) occurs when a peer reviews the code, often at the software developer’s request, before it can be merged into a project.

Different types of work product reviews (e.g., formal, and informal) are distinguished by purpose, level of independence, tools and techniques used, roles involved, and by the subject of the activity. Reviews play important roles in software quality, in SCM, and in the sharing of knowledge among colleagues. However, these different roles share a single purpose — to ensure the quality of the delivered products. Reviews should be part of the software engineering culture and should be planned, executed, and documented during the software life cycle. In Agile life cycles, pair programming invites continuous reviews. Different review types for work products are described in ISO/IEC 20246:2017 [12]:

- Ad hoc reviews — unstructured reviews where each reviewer is expected to find as many defects as possible of any type;
- Checklist-based reviews — systematic reviews identifying issues based on checklists;
- Scenario-based reviews — reviews where reviewers are provided with structured guidelines on how to read through the work product under review;
- Perspective-based reviews — reviews where reviewers take on a specific technical perspective and review the work product from that stakeholder’s viewpoint; and
- Role-based reviews — reviews in which the reviewer evaluates the work product from the perspective of various stakeholder roles, which might differ from their daily role.

Audits are more formal activities that are often mandated to be performed by third parties to ensure independence. In mature organizations, technical reviews and audits are fully integrated with the overall project plans. Therefore, technical reviews and audits should be planned, approved, and conducted. Although a project audit often addresses the whole project’s current state, technical reviews can also be more focused and address a specific project phase [24]. System requirements reviews help ensure that the level of understanding of top-level system requirements is adequate to support further requirements analysis and design activities and that the system can proceed into initial system design with acceptable risk; System functional or preliminary design reviews help ensure that the system under review can proceed into preliminary or detailed design with acceptable risk and that all system requirements and functional performance requirements derived from the approved preliminary system specification are defined and consistent with the project budget, program schedule, risk, and other program and system constraints; Preliminary design reviews help ensure that the preliminary design for the system under review is sufficiently mature and ready to proceed into detailed design and can meet the stated performance requirements within program budget, schedule, risk and other program and system constraints; Test readiness reviews assess test objectives, test methods and procedures, test scope, safety, readiness for the project test and evaluation, and whether test resources have been properly identified and obtained; Production readiness reviews ascertain that the system design is ready for production and that the project has accomplished adequate production planning for entering production.

4. Software Quality Tools

[c3s2.3, c7s8.1, c7s11]

Software tools improve software quality. Simple tools can be forms and checklists (e.g., a requirements traceability matrix or a code

review checklist). But automated tools can also be of great help to improve software efficiency and quality. Examples of automated tools are tools that allow code versioning/branching (e.g., Git) and pull requests for code review. DevOps tools in services/scripts like on-demand environments, continuous integration/continuous delivery (CI/CD), code quality assessment, and automated testing are important contributors to software quality. (See the Software Operations KA about tools.)

These tools are known as static and dynamic analysis tools. Static analysis tools input source code, perform syntactical and semantic analysis without executing the code. There is a large variety in the depth, thoroughness and scope of static analysis tools that can be applied to artifacts, including models, and source code. (See the Software Construction, Software Testing, and Software Maintenance KAs for descriptions of dynamic analysis tools.) Categories of static analysis tools include the following:

- Tools that facilitate and partially automate reviews and inspections of documents and code. These tools can route work to different participants to partially automate and control the review process.

In addition, they allow users to enter defects found during inspections and reviews for later removal;

- Tools that help organizations perform software safety hazard analysis. These tools provide, for example, automated support for failure mode and effects analysis (FMEA) and fault tree analysis (FTA);
- Tools that support tracking of software problems. These tools enable entry of anomalies discovered during software testing and subsequent analysis, disposition, and resolution. Some tools include support for workflow and for tracking problem resolution status; and
- Tools that analyze data captured from software engineering environments and software test environments and produce visual displays of quantified data in graphs, charts, and tables. These tools sometimes include the functionality to perform statistical analysis on data sets (to discern trends and make forecasts). Some of these tools provide defect injection and removal rates, defect densities, yields, and distribution of defect injection and removal for each life cycle phase.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

Topic	Laporte and April 2018 [1*]	Sommerville 2016 [7*]	IEEE Software Code of Ethics [5*]	Wieggers 2013 [13*]
1. Software Quality Fundamentals				
1.1. Software Engineering Culture and Ethics	C1s6, C2s3		X	
1.2. Value and Cost of Quality	c2s2			
1.3. Standards, Models and Certifications	c4	c24s2		
1.4. Software Dependability and Integrity Levels	c4s8 c7s2-3	C10		

2. Software Quality Management Process				
2.1. Software Quality Improvement	C9s9			
2.2. Plan Quality Management				
2.3. Evaluate Quality Management	C10	c24s5		
2.4. Perform Corrective and Preventive Actions	c1s3			
3. Software Quality Assurance Process				
3.1. Prepare for Quality Assurance	c1s5, c4s6			c14
3.2. Perform Process Assurance	c3s2-3 c8, c9, c4s6.1.3	c25		
3.3. Perform Product Assurance	c3s2-3 c5, c7, c4s6.1.2	c4s1.2		
3.4. Verification & Validation and Tests	c5, c6, c7	c10s10.5		
4. Software Quality Tools	c3s2.3, c7s8.1, c7s11		X	

FURTHER READINGS

IEEE 730-2014, “IEEE Standard for Software Quality Assurance Processes,” 2014 [6].

Requirements for initiating, planning, controlling, and executing the Software Quality Assurance processes of a software development or maintenance project.

IEEE 1012:2016, “IEEE Standard for System, Software, and Hardware Verification and Validation,” 2016 [11].

Verification and validation (V&V) processes are used to determine whether the development products of a given activity conform to that activity’s requirements and whether the product satisfies its intended use and user needs. V&V life cycle process requirements are specified for different integrity levels.

ISO/IEC 20246:2017, “Software and Systems Engineering — Work Product Reviews,” 2017 [12].

Establishes a generic framework for work product reviews that can be referenced and used by all organizations involved in the management, development, testing and maintenance of systems and software.

N. Leveson, *Safeware: System Safety and Computers* [15].

This book describes the importance of software safety practices and how these practices can be incorporated into software development projects.

T. Gilb and D. Graham, *Software Inspection* [16].

This book introduces measurement and statistical sampling for reviews and defects. It presents techniques that produce quantified results for reducing defects, improving productivity, tracking projects and creating documentation.

K.E. Wiegers, *Peer Reviews in Software: A Practical Guide* [17*].

This book provides clear, succinct explanations of different peer review methods distinguished by level of formality and effectiveness. It provides pragmatic guidance for implementing the methods and for determining which methods are appropriate for given circumstances.

REFERENCES

- [1*] C.Y. Laporte and A. April, *Software Quality Assurance*, IEEE Press, 2018.
- [2] P.B. Crosby, *Quality Is Free*, McGraw-Hill, 1979.
- [3] W. Humphrey, *Managing the Software Process*, Addison-Wesley, 1989.
- [4] “ISO/IEC 25010:2011 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuRE) — Systems and Software Quality Models,” 2011.
- [5*] IEEE CS/ACM Joint Task Force on Software Engineering Ethics and Professional Practices, “Software Engineering Code of Ethics and Professional Practice <https://www.computer.org/education/code-of-ethics>.”
- [6] IEEE 730 Standard for Software Quality Assurance Processes, IEEE, 2014.
- [7*] I. Sommerville, *Software Engineering*, 10th ed., Addison-Wesley, 2016.
- [8] RTCA, “DO-178C, Software Considerations in Airborne Systems and Equipment Certification,” 2012. Also known as ED-12C in EUROCAE.
- [9] ISO/IEC 15026-1:2019 Systems and Software Engineering — Systems and Software Assurance — Part 1: Concepts and Vocabulary, 2019.
- [10] “ISO 9001:2015 Quality Management Systems — Requirements,” 2015.
- [11] IEEE Std. 1012:2016, “Standard for System and Software Verification and Validation,” 2016.
- [12] ISO/IEC 20246:2017, “Software and systems engineering — Work product reviews,” 2017.
- [13*] K.E. Wiegers, *Software Requirements*, 3rd ed., Microsoft Press, 2013.
- [14] ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary, 2017.
- [15] N. Leveson, *Safeware: System Safety and Computers*, Addison-Wesley Professional, 1995.
- [16] T. Gilb and D. Graham, *Software Inspection*, Addison-Wesley Professional, 1993.
- [17*] K.E. Wiegers, *Peer Reviews in Software: A Practical Guide*, Addison-Wesley Professional, 2001.
- [18] BS EN 50128:2011+A2:2020, “Standard for Railway Applications – Communications, Signaling and Processing Systems – Software for Railway Control and Protection Systems,” British-Adopted European Standard, 10 August 2020.
- [19] K. Iberle, *They don’t care about quality*, proceedings of STAR East, Orlando, United States, 2013, available at <https://kiberle.com/publications/>.
- [20] D. Wallace, L.M. Ippolito, and B.B. Cuthill, *Reference Information for the Software Verification and Validation Process*, National Institute of

- Standards and Technology (NIST), U.D. Department of Commerce, Special Publication 500-234, 1996.
- [21] IEC 60300-1:2014, “Dependability Management — Part 1: Guidance for Management and Application,” 2014.
 - [22] D. Leffingwell, *SAFe 4.5 Reference Guide: Scaled Agile Framework For Lean Enterprises*, 2nd ed., Addison-Wesley, 2018.
 - [23] ISO/IEC TS 33061:2021, “Information technology — Process assessment — Process Assessment Model for Software Life Cycle Processes,” 2021.
 - [24] IEEE 15288.2:2014, “IEEE Standard for Technical Reviews and Audits on Defense Programs,” 2014.
 - [25] A guide to the Project management Body of Knowledge, 7th edition, PMI, 2021.
 - [26] ISO/IEC/IEEE 90003:2018, “Guidelines for the application of ISO 9001:2015 to computer software”, 2018.
 - [27] COBIT, “Control Objectives for Information Technology”, version 2019, ISACA and the IT Governance Institute.
 - [28] BABOK, “A guide to the Business Analysis Body of Knowledge”, version 3, International Institute of Business Analysis, 04-2015.
 - [29] CMMI, “Capability Maturity Model Integration”, version 10, ISACA, 2023.
 - [30] TOGAF, “Open Group Architecture Framework”, version 10, 2022.
 - [31] ISO/IEC 27001:2022, “Information security, cybersecurity, and privacy protection — Information security — management systems — Requirements”, 2022.
 - [32] Humphrey, W., .PSP: A Self-Improvement Process for Software Engineers, Addison-Wesley Professional, 2005.